

JSAN CTS DATA CENTER OPSDESK

Developer Build & Architecture Specification

Document Item	Value
Version	1.0
Status	Build baseline / architecture specification
Audience	Developers, QA, Infra/NOC, Delivery, Product Owner
Primary objective	Build a data-center-specific ITSM + CMDB + monitoring integration platform for CTS/ISAN operations
Date	01 September 2026

Important for the team — This is not a ServiceNow clone. Build only the capabilities required for CTS data-center operations, and reuse mature monitoring/logging technologies instead of rebuilding them.

1. How to Use This Document

This document is intentionally written for developers who may be new to IT Service Management (ITSM), data-center operations, CMDB concepts, monitoring, Dell/HPE hardware management, and SLA workflows. Read the sections in order before coding.

1. Understand the business problem and terminology first.
2. Use the architecture and design principles as non-negotiable guardrails.
3. Build the MVP in the sequence defined in the delivery plan; do not start with AI, microservices, or vendor automation.
4. Use the data model and state machines exactly as the baseline unless an architecture decision is approved.
5. Every feature must include RBAC, audit events, validation, tests, and operational logging before it is considered complete.

Golden rule — The system of record is the OpsDesk application; monitoring tools provide telemetry, but they do not own incidents, CMDB relationships, SLA decisions, worklogs, vendor cases, or the operational audit trail.

2. Business Problem and Product Vision

CTS data-center support currently requires a clear mechanism to see infrastructure health, record technical and hardware issues, assign ownership, track work and escalation, manage vendor interactions, and demonstrate the full operational timeline. Email, spreadsheets, chat messages and standalone monitoring screens cannot provide one authoritative view.

2.1 Product Vision

Product statement — JSAN CTS Data Center OpsDesk is a centralized data-center infrastructure operations and service-management platform that combines site/asset visibility, CMDB, incidents and requests, SLA governance, engineer worklogs, Dell/HPE hardware lifecycle tracking, monitoring alerts, vendor/RMA coordination, SOPs, risk/BCP records, and management reporting.

2.2 End-to-End Operating Flow

The platform must support this operating chain:

Site / Data Center → Rack → Asset / CI → Health / Alert → Incident → Engineer Action → Vendor Case / RMA → Restoration → RCA → SLA / Management Report

Example: a server disk enters predictive failure. Monitoring detects the event, the platform correlates it to the server CI, opens or updates a P2 incident, routes it to the owning team, starts the SLA timer, captures engineering effort, records the Dell/HPE case and replacement part, confirms health recovery, and closes the incident with a complete audit timeline.

3. Scope: What We Build and What We Do Not Build

3.1 In Scope

- Command Center showing site and infrastructure health.
- Site, rack, server, network, power and hardware asset inventory / CMDB.
- Incident management, service requests, problems/RCA and planned changes.

- Priority, routing, ownership, escalation and configurable SLA calendars.
- Engineer clocking/worklogs, status timestamps and immutable audit trail.
- Monitoring alert ingestion and automatic incident creation/update.
- Dell and HPE hardware inventory/health using Dell OME/iDRAC and HPE iLO/Redfish where available.
- Vendor support case, warranty, RMA, replacement part and local service-provider tracking.
- Site contacts, on-call roster, RACI/ownership, SOP/knowledge, maintenance, risks and BCP information.
- Role-based CTS and JSAN views, reports and exports.

3.2 Explicit Non-Goals for MVP

- Do not build a custom time-series monitoring database. Use Zabbix/Prometheus/Grafana-class tools.
- Do not build a custom log search engine. Use Loki/OpenSearch-class tools.
- Do not build a full ServiceNow clone, HR service desk, CRM, procurement suite or enterprise ERP.
- Do not implement arbitrary remote shell/PowerShell execution from the portal in v1.
- Do not expose iDRAC/iLO management ports to the public Internet.
- Do not start with Kubernetes or many microservices; begin with a modular monolith.
- Do not add AI until ticket, CMDB and telemetry data are stable and trustworthy.

4. Users, Roles and Access Model

Role	Primary Responsibilities	Typical Access
Super Admin	Platform configuration, tenants, integrations, roles	All data and settings
Service Desk / NOC Engineer	Triage, acknowledge, route, update incidents	Assigned/all permitted sites; incident and alert operations
Site Engineer	On-site/remote diagnosis, worklogs, evidence, restoration	Assigned sites/assets; limited admin
Infrastructure Lead	Ownership, change approval, escalation, technical review	Infrastructure and support modules
Vendor Coordinator	Dell/HPE/ISP/local vendor cases, RMA and dispatch	Vendor cases and linked incidents/assets
Delivery / Operations Manager	SLA, workload, escalation, risk and reporting	Cross-site dashboards and reports
CTS Manager / Viewer	Customer visibility, incident status, SLA and reports	Only authorized customer/site scope
Auditor / Read Only	Evidence and history	Read-only; no operational changes

RBAC rule — Authorization must be enforced in the backend on every API call. Hiding a button in the UI is not security.

5. Architecture Overview

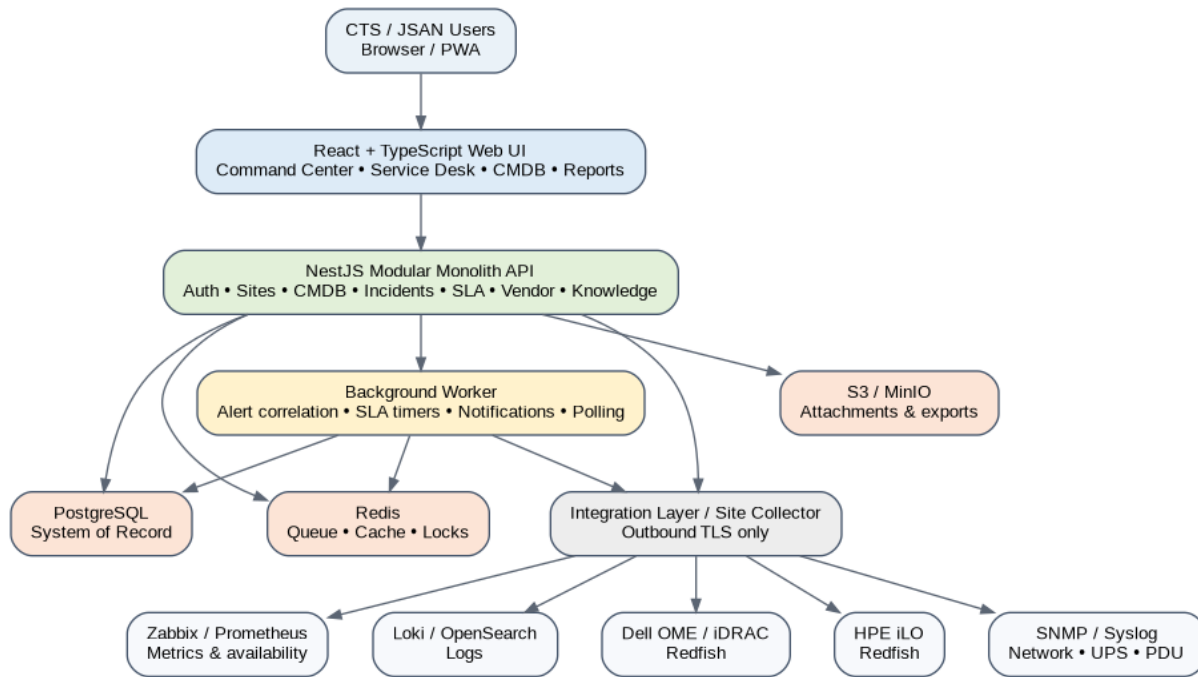


Figure 1 — Recommended logical architecture

5.1 Architecture Decision: Modular Monolith First

The backend should be one deployable NestJS application organized into strongly separated domain modules. This gives the fresher team one codebase and one deployment while preserving boundaries that can later be split if scale requires it.

- One API service initially; modules cannot directly manipulate another module's database logic.
- Use service interfaces/events between modules.
- Background jobs run as a separate worker process from the same codebase.
- External collectors/integrations are isolated from core ticket logic.
- Keep telemetry data out of PostgreSQL except normalized alerts, current state and references.

6. Design Principles — Non-Negotiable

Principle	Meaning for Developers
CMDB-first	Every infrastructure incident, alert, maintenance activity and vendor case should link to a Configuration Item whenever possible.
Configuration over hard-code	SLA times, support hours, priorities, ownership, site calendars and alert rules must be configurable in tables/admin screens.
Audit everything	Every state change, assignment, SLA pause, worklog, attachment, vendor update and admin change creates an append-only audit event.
Idempotent integrations	Repeated webhooks/polls must not create duplicate incidents. Use external_event_id and alert fingerprints.

Principle	Meaning for Developers
Separate telemetry from ITSM	Metrics/logs belong in monitoring/log stores; OpsDesk stores summarized state and operational records.
Secure management plane	Collectors connect outbound over TLS/mTLS. Never publish iDRAC/iLO/SNMP endpoints to the Internet.
Least privilege	Users, collectors and API tokens get only the permissions required for their role/site.
Graceful degradation	If monitoring or a vendor API is down, users must still be able to create/update tickets and see the last known state.
Human override with traceability	Automation may suggest or route, but an authorized engineer can override priority/owner with a mandatory reason recorded in audit.
No premature complexity	Prefer clear REST APIs, Postgres, Redis and Docker. Introduce event buses, Kubernetes and AI only when justified.

7. Recommended Technology Stack

Layer	Recommended Baseline	Why
Frontend	React + TypeScript + Vite + Material UI (or equivalent component library)	Fast development, typed UI, accessible enterprise components
Backend	NestJS + TypeScript	Strong module boundaries, validation, OpenAPI support, suitable for junior developers
Database	PostgreSQL	Relational integrity is important for CMDB, incidents, SLA, ownership and reports
ORM	Prisma or TypeORM (select one and standardize)	Migrations, typed models and safer queries
Queue / Cache	Redis + BullMQ	SLA timers, integrations, background jobs, cache and distributed locks
Object Storage	S3-compatible storage / MinIO for development	Attachments, evidence, exported reports
Monitoring	Zabbix and/or Prometheus + Grafana	Do not rebuild metrics collection/visualization
Logs	Loki or OpenSearch	Central search and retention for technical logs
Hardware APIs	Redfish; Dell OME/iDRAC; HPE iLO	Vendor-neutral standard where possible; vendor-specific enrichments where useful
Auth	OIDC/OAuth2 via enterprise IdP; Keycloak for dev/non-enterprise environments	MFA/SSO capable and avoids custom password security
Packaging	Docker + Docker Compose initially	Reproducible developer/staging deployments
CI/CD	GitHub Actions / GitLab CI / equivalent	Lint, test, scan, build and deploy consistently

8. Suggested Repository Structure

```
cts-dc-opsdesk/  
├── apps/  
│   ├── web/           # React UI  
│   ├── api/           # NestJS HTTP/WebSocket API  
│   └── worker/        # queues, SLA, alerts, notifications  
├── packages/  
│   ├── shared-types/  
│   ├── validation/  
│   ├── ui-components/  
│   └── observability/  
├── integrations/  
│   ├── redfish/  
│   ├── dell-ome/  
│   ├── zabbix/  
│   ├── prometheus/  
│   ├── snmp/  
│   └── email/  
├── infra/  
│   ├── docker/  
│   ├── migrations/  
│   └── deployment/  
├── docs/  
│   ├── api/  
│   ├── adr/  
│   └── runbooks/  
├── tests/  
│   ├── integration/  
│   └── e2e/
```

Team discipline — Do not create a “utils” folder containing business logic. Put logic inside the owning domain module with tests.

9. Core Domain and CMDB Model

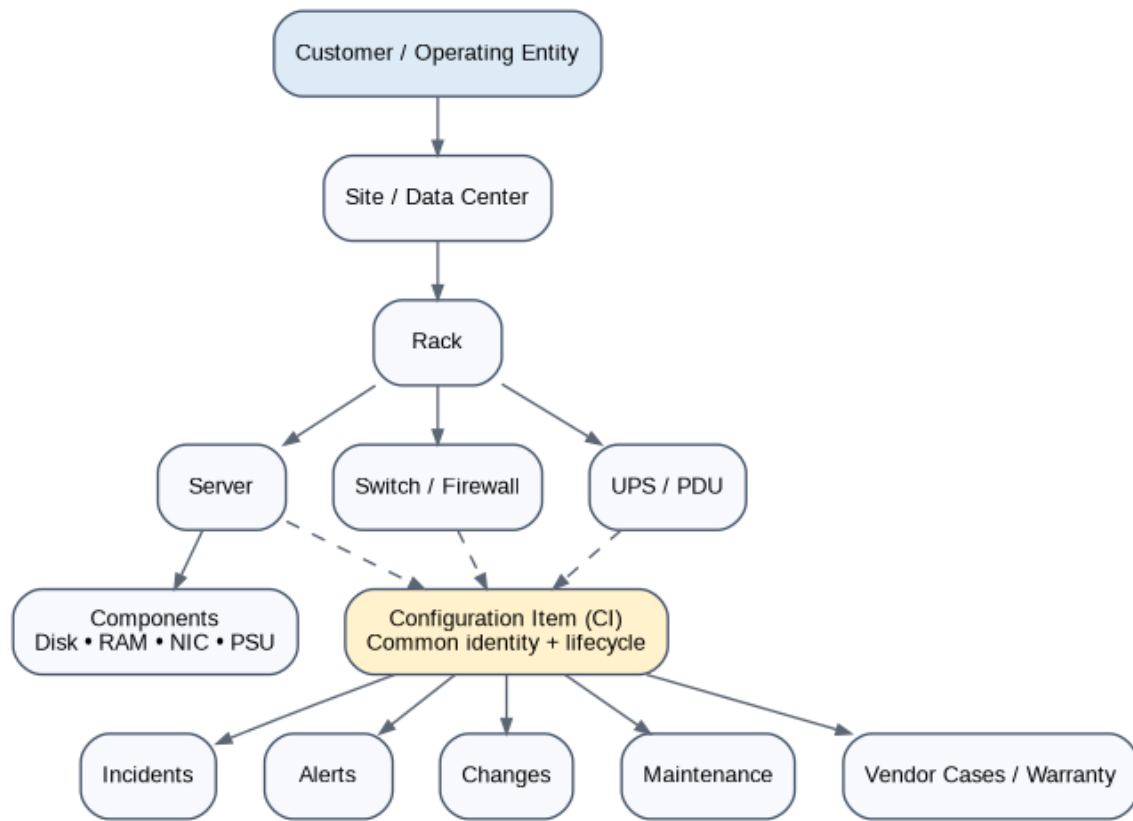


Figure 2 — Simplified CMDB relationship model

9.1 Configuration Item (CI)

A Configuration Item is any managed infrastructure component whose health, changes, incidents or lifecycle matters. Examples: server, firewall, switch, UPS, PDU, storage system, VM, critical ingestion service, or WAN/ISP circuit.

Field	Required?	Example / Rule
id	Yes	UUID
ci_code	Yes	Stable human-readable code such as SITE-R01-SRV-038
site_id	Yes	Foreign key to site
ci_type	Yes	SERVER / FIREWALL / SWITCH / UPS / PDU / STORAGE / SERVICE / CIRCUIT
name	Yes	Display name
manufacturer	When applicable	Dell / HPE / Cisco / APC / etc.
model	When applicable	Vendor model
serial_or_service_tag	When applicable	Unique hardware identity
management_address	Restricted	Encrypted or access-controlled; never expose to customer viewer
owner_group_id	Yes	Primary support group

Field	Required?	Example / Rule
managed_by	Yes	JSAN / CTS / SHARED / VENDOR
criticality	Yes	CRITICAL / HIGH / MEDIUM / LOW
lifecycle_status	Yes	PLANNED / ACTIVE / MAINTENANCE / RETIRED
metadata	Optional	JSONB for vendor-specific fields only

9.2 CMDB Relationship Examples

- SITE contains RACK.
- RACK contains SERVER / SWITCH / UPS / PDU.
- SERVER contains DISK / NIC / PSU / MEMORY components.
- SERVICE runs_on SERVER or VM.
- SERVER depends_on SWITCH and UPS.
- SITE uses ISP_CIRCUIT.
- INCIDENT affects one or more CIs.

10. Functional Module Requirements

10.1 Command Center

The landing page is an operations dashboard, not a ticket inbox. It should answer “What is unhealthy, where, who owns it, and what needs attention now?”

- Global counters: sites healthy/warning/critical, servers reachable, critical alerts, P1/P2 incidents, SLA-at-risk incidents.
- Site cards: current health, server availability, network/power status, open incidents, oldest unresolved issue.
- Operational queues: unassigned, awaiting acknowledgement, SLA breach risk, vendor waiting, reopened incidents.
- Health status based on configurable roll-up rules; never infer “healthy” only because no ticket exists.
- All dashboard numbers must be clickable to filtered detail views.

10.2 Sites and Infrastructure

- Site profile: code, name, timezone, support calendar, address (restricted as required), ownership, status and contacts.
- Rack and CI inventory with search, filters, bulk import and lifecycle status.
- Current health snapshot and last heartbeat for monitorable CIs.
- Linked incidents, changes, maintenance, risks, warranties and vendor cases.
- Separate “managed by JSAN”, “managed by CTS” and shared views; do not mix RACI ownership.

10.3 Incident Management

Requirement	Implementation Detail
Create incident	Manual, email/API or monitoring automation. Generate a unique incident number.
Required context	Site, affected CI when known, category, impact, urgency, priority, owner group, short description.

Requirement	Implementation Detail
Status model	NEW → ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → PENDING_* → RESOLVED → CLOSED; support REOPEN.
Timeline	Append-only events for status, owner, comments, SLA, worklog, vendor and attachment activity.
Evidence	Attachments stored in object storage; metadata and hash stored in DB.
Relationships	Link parent/child incidents, duplicate, problem, change, vendor case and related alerts.
Closure	Resolution category, root cause summary, restoration time and closure notes mandatory for selected priorities.

10.4 Service Requests

Service requests are planned asks, not failures. Examples: new access, hardware installation, scheduled firmware work, account change, rack activity or planned configuration. Requests need approval steps and fulfillment tasks but should not be mixed with incident SLA reporting.

10.5 Problems and RCA

- Create a problem record for repeated incidents or major incidents requiring formal RCA.
- Capture symptoms, known error, root cause, corrective action, preventive action, owner and due date.
- Link all related incidents and changes.
- Track action items to completion; closing an incident must not automatically close the problem.

10.6 Change Management

- Change types: standard, normal and emergency.
- Fields: reason, affected CIs, implementation plan, validation plan, rollback plan, risk, window, approver, outcome.
- Approved change windows can suppress or annotate expected monitoring alerts.
- Emergency change requires post-implementation review.

10.7 Engineer Clocking / Worklogs

“Clocking and logging” has two separate meanings and both must be built: engineer effort tracking and technical/audit event logging.

Worklog Field	Rule
worklog_id	UUID, immutable identity
incident_id	Required for incident work
engineer_id	Authenticated user
activity_type	REMOTE_WORK / ONSITE / TRAVEL / VENDOR_CALL / TESTING / WAITING / OTHER
started_at / ended_at	Timezone-aware UTC storage; display in site/user timezone
duration_minutes	Derived by backend; no negative duration
billable_or_contract_flag	Configurable; not assumed
notes	Required for manual time entry or edits

Worklog Field	Rule
edit_reason	Mandatory if authorized user corrects a submitted worklog
audit_event	Created for every create/update/correction

Do not implement a simple mutable stopwatch — Operational evidence must preserve who changed time, from what value, to what value, when, and why.

10.8 SLA and Escalation Engine

SLA must be policy-driven. The Saturday MOM indicates different handling for business hours, after-hours and emergency cases; therefore never hard-code one global clock.

Concept	Required Behavior
Support calendar	Per site/service: timezone, workdays, business hours, holidays and 24x7 flag.
Acknowledgement SLA	Starts at qualifying incident creation; stops at accepted/acknowledged event.
Resolution SLA	Runs until resolution; may pause only in policy-approved waiting states.
Escalation	Notify next level at configurable thresholds such as 50%, 75%, 90%, breach.
Priority	Calculated from impact × urgency matrix, but authorized override allowed with reason.
Breach evidence	Store exact timer events and policy version used.

Priority	Illustrative Ack Target	Illustrative Restore/Resolve Target	Clock
P1 Critical	15 min	4 hours	Typically 24x7
P2 High	30 min	8 hours	Policy-defined
P3 Medium	4 business hours	2 business days	Business calendar
P4 Low	1 business day	5 business days	Business calendar

These are engineering defaults only — CTS/JSAN must confirm contractual SLA values, support hours and pause rules before production. Store them as configurable policies with effective dates.

10.9 Monitoring and Alert Management



Figure 3 — Alert-to-incident lifecycle and vendor escalation

Monitoring integrations produce normalized alert events. The rule engine decides whether to create a new incident, update an existing incident, suppress an expected event, or only record telemetry.

Normalized Alert Field	Example
source	ZABBIX / PROMETHEUS / REDFISH / SNMP / SYSLOG
external_event_id	Provider event identifier
site_id / ci_id	Resolved CMDB context
alert_type	DISK_PREDICTIVE_FAILURE
severity	CRITICAL / HIGH / WARNING / INFO
fingerprint	Hash of stable issue identity used for deduplication
first_seen_at / last_seen_at	Event timeline
state	OPEN / ACKNOWLEDGED / RECOVERED
raw_reference	Pointer/reference, not unlimited raw payload storage

10.10 Alert Deduplication Rules

1. Build a fingerprint from site + CI + alert type + stable component identifier.
2. If an OPEN incident already exists for the fingerprint, append the event and update last_seen_at instead of creating a duplicate.
3. If the alert recovers, mark the alert recovered; do not automatically close the incident unless the policy explicitly permits it.
4. If the same alert flaps repeatedly inside a configurable window, count occurrences and surface “flapping” instead of generating multiple incidents.
5. During an approved maintenance window, suppress auto-ticketing or label the alert as expected according to rule configuration.

10.11 Log Management

- Use Loki/OpenSearch for high-volume logs; do not copy all logs into PostgreSQL.
- OpsDesk stores log-source registration, query links, extracted evidence and incident-linked snippets/references.
- Redact credentials, tokens, personal data and secrets before logs are displayed or attached.
- Define retention separately for infrastructure logs, audit logs and incident evidence.

10.12 Dell and HPE Hardware Management

Hardware integration should use Redfish as the common baseline, then add Dell/HPE-specific data when available. The portal should present one normalized server health model even if raw vendor schemas differ.

Capability	Dell	HPE	OpsDesk Normalized Output
Hardware identity	Service Tag / OME inventory	Serial / iLO inventory	manufacturer, model, serial/service tag
Health	iDRAC/Redfish / OME	iLO/Redfish	overall health, component health
Power	Redfish power data	Redfish power data	power state, PSU health
Storage	Controller/drive health	Controller/drive health	RAID/storage state, drive health
Thermal	Sensors	Sensors	temperature/fan warnings
Firmware	Inventory	Inventory	firmware versions and last refresh
Support tracking	Warranty/vendor case data where permitted	Warranty/vendor case data where permitted	warranty + vendor case/RMA record

Safety boundary — The first release is read/observe/record. Do not enable firmware updates, remote reboot, BIOS changes or other destructive hardware actions from the portal without a separately reviewed control design.

10.13 Vendor / RMA Management

Field	Purpose
vendor_case_no	Dell/HPE/ISP/local vendor support reference
linked_incident_id	Operational issue that required vendor action
ci_id	Affected asset/component
warranty_status	Active / expired / unknown
opened_at / acknowledged_at	Vendor response tracking
rma_required	Boolean
replacement_part	Part name / part number if available
dispatch_status	Requested / approved / shipped / delivered / installed / returned
vendor_eta	Expected action/arrival
vendor_updates	Append-only event history
closed_at / outcome	Completion evidence

10.14 SOP / Knowledge

- Site-specific SOPs and global runbooks.
- Article versioning, owner, approval state and review date.
- Link knowledge articles to incident category and CI type.
- Search by symptom, error, vendor, site and hardware component.
- Do not allow draft/unapproved procedures to be shown as authoritative runbooks.

10.15 Risk and BCP

- Risk register for missing redundancy, hardware single points of failure, ISP dependency, power backup and support gaps.
- Fields: likelihood, impact, score, mitigation, owner, due date, status and evidence.
- BCP plan per site/service with recovery strategy, alternate site, RTO, RPO, contacts, last test date and next test date.
- Do not label the operating model “zero downtime” where physical redundancy does not exist; report target availability and residual risk.

10.16 Reports

- Availability by site and CI type.
- Incident volume by priority/category/site/vendor.
- MTTA, MTTR and SLA attainment.
- Top recurring failures and problem records.
- Vendor response, RMA duration and repeat hardware failure.
- Engineer effort and on-site vs remote activity.

- Asset warranty expiries and maintenance due.
- Risk/BCP readiness and overdue mitigations.

11. Deployment and Site Collector Design

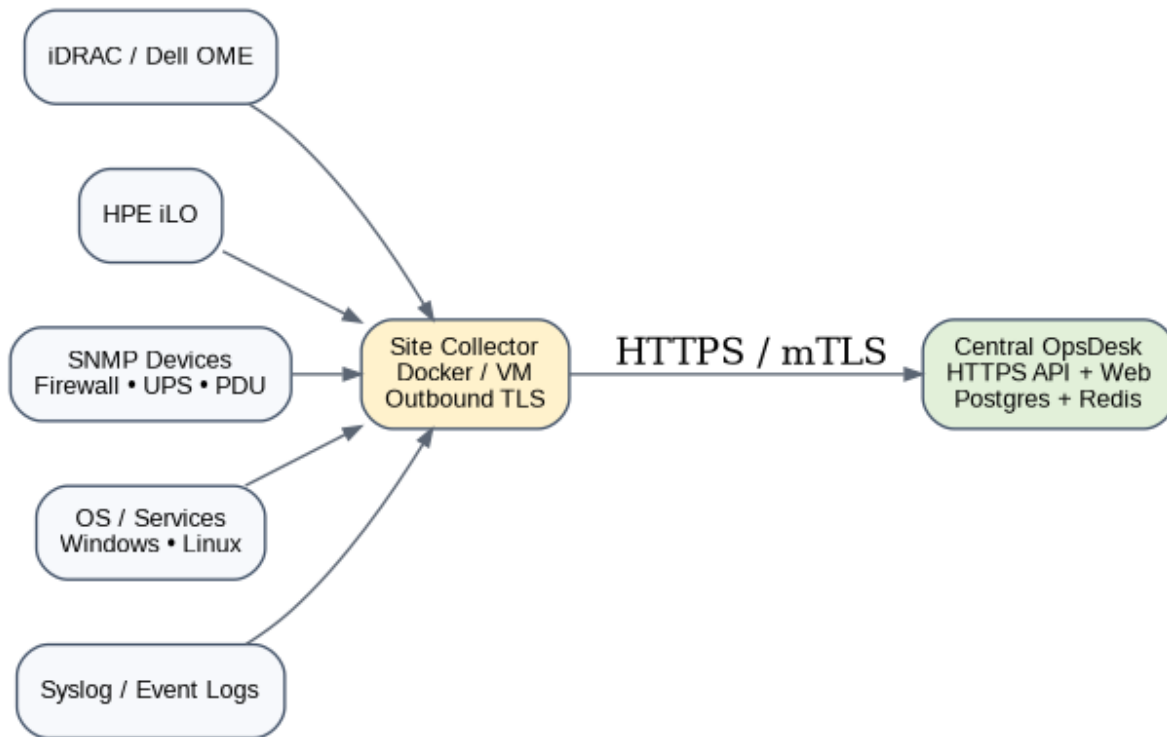


Figure 4 — Secure site collector pattern

11.1 Why a Site Collector?

Infrastructure management interfaces should remain inside the local site network. A lightweight collector can query local APIs and devices and send normalized observations outbound to the central platform over secure HTTPS/mTLS. This avoids inbound firewall openings to iDRAC/iLO and makes intermittent connectivity manageable.

11.2 Collector Responsibilities

- Discover or query approved local endpoints from explicit configuration.
- Poll health/inventory at configured intervals.
- Receive local syslog/SNMP traps when enabled.
- Normalize vendor payloads into a stable event schema.
- Buffer events locally if the central service is temporarily unreachable.
- Use unique event IDs and idempotent upload to prevent duplicates after reconnect.
- Never store central user credentials; use scoped machine credentials from a secret store.

11.3 Suggested Polling Defaults

Data Type	Initial Interval	Notes
Reachability / heartbeat	60 seconds	Tune by criticality and network overhead

Data Type	Initial Interval	Notes
Critical hardware health	1–5 minutes	Prefer event subscription/webhook where supported
Inventory	6–24 hours	Inventory does not need minute-level polling
Warranty	Daily/weekly	Dependent on vendor API/entitlement
Logs	Streaming/batched	Handled by log platform rather than relational DB

12. Backend Module Boundaries

Module	Owns	Must Not Own
auth	identity mapping, sessions/tokens, roles	incident business rules
sites	sites, timezone, contacts, support calendars	hardware telemetry
cmdb	CI, components, relationships, lifecycle	ticket SLA state
incidents	incident state machine, assignments, comments	vendor polling
sla	policy versions, timers, escalations	UI-only countdowns
alerts	normalized alerts, fingerprints, correlation	raw time-series storage
worklogs	engineer activity and time corrections	authentication
vendors	vendors, warranties, cases, RMA	monitoring metrics
knowledge	SOPs/runbooks, approvals, versions	incident creation
changes	change workflow and maintenance windows	monitoring storage
risks	risk register, BCP records	ticket state
reports	read models/aggregations	source-of-truth mutations
audit	append-only audit records	business entity edits

13. Database Baseline

Use PostgreSQL migrations from the first commit. Never create production schema manually. All timestamps are stored in UTC with timezone-aware types and rendered in the user/site timezone.

Table / Entity	Purpose
users	User profile / IdP subject / status
roles / permissions	RBAC model
sites	Data-center master
site_contacts	Organizational and escalation contacts
support_calendars	Business hours and holidays
support_groups	Ownership/assignment groups
vendors	Dell/HPE/ISP/local vendors

Table / Entity	Purpose
vendor_contracts	Support entitlement and coverage
configuration_items	CMDB master
ci_relations	CI dependency/containment graph
ci_components	Disks/PSUs/NICs/etc. when tracked separately
warranties	Asset warranty/support metadata
health_snapshots	Latest normalized health only
alerts	Normalized operational alerts
incidents	Incident header/current state
incident_events	Append-only incident timeline
incident_comments	Public/internal comments
worklogs	Engineer clocking and effort
sla_policies	Versioned SLA configuration
sla_instances	Per-ticket timer state
service_requests	Request header
request_tasks	Fulfillment tasks
problems	RCA/problem records
changes	Change records
maintenance_windows	Approved suppression windows
vendor_cases	Vendor support/RMA cases
knowledge_articles	Versioned SOPs/runbooks
risks	Infrastructure risks
bcp_plans	BCP and recovery targets
attachments	Object metadata/hash/access
notifications	Delivery state
audit_events	Append-only compliance trail

13.1 Data Rules

- Use UUID primary keys internally and separate human-readable numbers/codes for UI.
- Use foreign keys for core relationships; do not store important IDs only inside JSON.
- Use JSONB only for vendor-specific or extensible metadata.
- Avoid hard delete for incidents, worklogs, vendor cases and audit records.
- Master records may be retired/disabled rather than deleted when referenced historically.
- Use optimistic locking/version columns on high-contention records such as incidents if required.
- Every mutation records actor, timestamp, request/correlation ID and before/after summary in audit.

14. API Design

14.1 API Conventions

- REST JSON under /api/v1.
- OpenAPI/Swagger generated from backend DTOs.
- All input validated server-side; reject unknown or invalid enum values.
- Pagination on every list endpoint; never return unlimited tickets/assets.
- Use request IDs/correlation IDs in response headers and logs.
- Idempotency key for external event ingestion and selected create operations.
- Consistent error shape: code, message, field_errors, correlation_id.
- Use SSE/WebSocket only for live dashboard updates; core operations remain REST.

Method	Endpoint	Purpose
GET	/api/v1/sites	List authorized sites
GET	/api/v1/sites/{id}/health	Site health roll-up
GET	/api/v1/cis	Search/filter CIs
GET	/api/v1/cis/{id}	CI detail and current state
POST	/api/v1/incidents	Create incident
GET	/api/v1/incidents	Incident queue/search
PATCH	/api/v1/incidents/{id}	Allowed field updates
POST	/api/v1/incidents/{id}/transition	Validated status transition
POST	/api/v1/incidents/{id}/worklogs	Add engineer worklog
POST	/api/v1/incidents/{id}/comments	Add comment
POST	/api/v1/alerts/ingest	Collector/monitoring event ingestion
POST	/api/v1/vendor-cases	Create vendor tracking record
POST	/api/v1/changes	Create change
GET	/api/v1/reports/sla	SLA report
GET	/api/v1/audit	Authorized audit search

14.2 Example Normalized Alert Payload

```
{
  "eventId": "site01-redfish-438832",
  "source": "REDFISH",
  "siteCode": "SITE01",
  "ciCode": "SITE01-R01-SRV-038",
  "alertType": "DISK_PREDICTIVE_FAILURE",
  "severity": "HIGH",
  "componentKey": "drive-bay-4",
  "occurredAt": "2026-09-01T08:01:15Z",
  "state": "OPEN",
  "summary": "Drive bay 4 reports predictive failure",
  "attributes": {
    "vendor": "HPE",
    "health": "Warning"
  }
}
```

15. Incident State Machine and Business Rules

From	To	Who Can Do It	Required Validation
NEW	ASSIGNED	System / Service Desk	Owner group/user resolved
ASSIGNED	ACKNOWLEDGED	Assigned engineer / lead	Acknowledged timestamp stored; ack SLA stops
ACKNOWLEDGED	IN_PROGRESS	Engineer	Optional initial diagnostic note by policy
IN_PROGRESS	PENDING_VENDOR	Engineer / lead	Vendor case or pending reason required
IN_PROGRESS	PENDING_CUSTOMER	Authorized support user	Reason required; SLA pause depends on policy
PENDING_*	IN_PROGRESS	Owner/authorized user	Resume timer if previously paused
IN_PROGRESS	RESOLVED	Engineer / lead	Resolution category + note; health check for monitored CI
RESOLVED	CLOSED	Service Desk / auto-policy	Closure validation
RESOLVED	REOPENED	Authorized user / monitoring rule	Reason required; SLA behavior policy-driven
Any open	CANCELLED	Lead/admin	Cancellation reason; never delete

State transitions are backend rules — Do not allow the frontend to set incident.status directly. Use a transition service that validates role, current state, required fields, SLA and audit events.

16. Priority, Impact and Urgency

Priority should normally be calculated from an impact × urgency matrix. The exact matrix is configurable. A sample baseline:

Impact \ Urgency	High	Medium	Low
High	P1	P2	P3
Medium	P2	P3	P3
Low	P3	P3	P4

- Impact examples: entire site/service unavailable; multiple servers; one critical CI; single non-critical component.
- Urgency examples: active outage; imminent failure; degraded but stable; informational/request.
- Authorized priority override requires a reason and generates an audit event.

17. Security Architecture

Control	Requirement
Identity	OIDC/OAuth2; MFA when supported; no custom weak password store in production.
Authorization	RBAC plus site/customer scope enforced server-side.
Transport	TLS for all central traffic; mTLS recommended for collectors.

Control	Requirement
Secrets	Secret manager/environment-injected secrets; never commit credentials or vendor API tokens.
Management interfaces	iDRAC/iLO/SNMP remain local; no direct Internet exposure.
Database	Encrypted storage/backups where platform supports it; least-privilege DB roles.
Attachments	Malware scan where available; content-type/size limits; signed access URLs.
Logs	Never log passwords, tokens, session IDs, private keys or full sensitive payloads.
Audit	Append-only audit trail for security and operational mutations.
Rate limiting	Protect login, search, event ingestion and webhook endpoints.
Dependency security	Automated dependency/SAST/container scanning in CI.

18. Non-Functional Engineering Targets

Area	Initial Engineering Target / Rule
Availability	Design central application for high availability later; MVP must support backup/restore and graceful restart. Do not promise zero downtime.
Performance	Common list/detail screens should normally return in <2 seconds under expected load; dashboards may load progressively.
Scale	Design relational queries/indexes for thousands of CIs and large incident history; keep high-volume metrics/logs external.
Reliability	Idempotent event ingestion and retries with exponential backoff.
Time zones	Store UTC; display site/user local time; SLA calendar calculates in site/policy timezone.
Accessibility	Keyboard support, labels, contrast and text/icon status; do not rely only on color.
Retention	Policy-configurable by data class: incidents/audit/evidence/telemetry/logs.
Backup	Automated DB + object storage backup with periodic restore test.
Observability	Structured logs, API latency/error metrics, queue depth, job failures, DB health, integration health.

19. UI / UX Design Rules

- The first screen is a Command Center with actionable health and SLA information.
- Use consistent severity/status labels and icons: Healthy, Warning, Critical, Unknown, Maintenance.
- Always show “last updated” for live/near-live health data.
- Incident detail uses one ordered timeline so users can understand what happened without reading multiple tabs.
- Separate internal engineer notes from customer-visible comments.
- Never expose raw vendor API payloads to normal users; normalize and progressively disclose technical details.
- Use filters that can be bookmarked/shared through URL query parameters.
- Tables require server-side pagination, sort and filter for large datasets.
- Confirmation dialogs are mandatory for risky state changes such as close/cancel/approve emergency change.

20. DevOps and Environments

Environment	Purpose	Rules
Local	Developer coding/testing	Docker Compose; seeded fake sites/assets; no real production credentials
DEV	Shared integration	Auto-deploy from development branch; synthetic monitoring data
UAT / STAGING	Business/infra validation	Production-like configuration; masked/synthetic data unless approved
PROD	CTS/JSAN operations	Protected branch/tag, approved migration, backup, rollback plan and monitored release

20.1 CI Pipeline Minimum

1. Install dependencies with lockfile.
2. Lint and formatting check.
3. TypeScript type check.
4. Unit tests.
5. Database migration validation.
6. Integration/API tests for critical modules.
7. Dependency and secret scan.
8. Container build and image scan.
9. Deploy to DEV only after all gates pass.

21. Testing Strategy

Test Type	What to Cover
Unit	Priority matrix, SLA calendar math, state transitions, alert fingerprint, permission helpers.
Integration	Postgres repositories, Redis jobs, object storage, event ingestion, vendor adapter mocks.
API	Validation, authorization, pagination, error shape, idempotency.
E2E	Create incident → assign → acknowledge → worklog → vendor → resolve → close.
Contract	Redfish/Dell/HPE adapter schemas using captured sanitized fixtures.
Security	RBAC/site isolation, token expiry, injection, unsafe upload, broken object authorization.
Resilience	Collector offline/reconnect, duplicate webhook, Redis restart, vendor API timeout, monitoring unavailable.
UAT	Operational scenarios executed by service desk/infra users with acceptance evidence.

21.1 Minimum Acceptance Scenarios

1. A service desk user manually creates a P2 incident linked to a server CI and the correct SLA starts.
2. A monitoring alert creates one incident; ten duplicate events update the same incident rather than create ten tickets.
3. A recovered alert is recorded, but the incident remains open until the configured operational rule is satisfied.
4. An engineer clocks remote work, moves the incident to Pending Vendor, and the timer behavior follows the policy.
5. A Dell/HPE hardware event is normalized to the correct server/component and visible on the asset page.
6. A vendor case and RMA part are linked to the incident and appear in the full timeline.
7. A CTS viewer cannot see another restricted site or internal-only engineer note.
8. Changing priority, owner, SLA state or worklog creates an audit entry showing actor, time and reason.
9. A planned maintenance window prevents expected alerts from creating unnecessary incidents according to rule configuration.
10. Backup restore produces a working staging instance with consistent incidents, CMDB relationships and attachments.

22. Phased Build Plan for a Fresher Team

The sequence below is more important than the dates. Do not begin hardware automation before ticketing, CMDB identity and audit are stable.

Phase	Indicative Focus	Exit Gate
0 — Foundation	Repo, Docker, auth, DB migrations, RBAC, audit, CI/CD, coding standards	Team can deploy a secure skeleton to DEV
1 — Operational MVP	Sites, contacts, CMDB basics, incidents, comments, attachments, worklogs	Manual incident lifecycle works end to end
2 — SLA + Command Center	Support calendars, SLA engine, escalation, dashboard, reports	Timers verified across timezone/business-hour cases
3 — Monitoring Integration	Alert ingest, correlation, dedup, health snapshots, maintenance suppression	Synthetic + one real monitoring source creates correct incidents
4 — Hardware Integration	Redfish, Dell/HPE adapters, component health/inventory	Approved server health visible without exposing management ports
5 — Vendor + Governance	Vendor cases/RMA, changes, problems/RCA, knowledge, risks/BCP	Major incident and RMA scenario validated
6 — Production Hardening	Performance, HA plan, security test, restore drill, runbooks, UAT	Formal go-live checklist signed off

23. Suggested 12-Sprint Backlog

Sprint	Build Focus	Key Deliverables
1	Foundation	Monorepo, Docker, NestJS/React skeleton, Postgres migrations, auth stub, CI
2	RBAC + Site Master	Roles, site scope, contacts, timezone, support group, audit framework
3	CMDB Core	CI CRUD, rack/server hierarchy, relationships, bulk import, search
4	Incident Core	Numbering, validation, status state machine, assignment, comments

Sprint	Build Focus	Key Deliverables
5	Worklogs + Attachments	Clocking, corrections/audit, S3/MinIO evidence, timeline
6	SLA	Policy versions, calendars, ack/resolve timers, escalation jobs
7	Command Center	Site health shell, incident queues, SLA risk, reports baseline
8	Alert Pipeline	Ingestion endpoint, fingerprint, dedup, correlation, recovery events
9	Monitoring Adapter	Zabbix/Prometheus integration, latest health, maintenance suppression
10	Redfish Hardware	Dell/HPE normalized inventory/health adapter with fixtures
11	Vendor/RMA + RCA	Vendor cases, RMA lifecycle, problem/RCA, change and knowledge basics
12	Hardening/UAT	Security, resilience, performance, backup restore, runbooks, UAT fixes

Planning note — If the team is very junior, treat a “sprint” as a delivery unit rather than promising a fixed calendar. Quality gates matter more than compressing the schedule.

24. Definition of Done

- User story acceptance criteria passed and demonstrated.
- Backend authorization and input validation implemented.
- Audit event emitted for business-critical mutations.
- Unit/integration tests added for new logic and edge cases.
- No TypeScript/lint errors and no critical security scan findings.
- Database migration reviewed; rollback/forward strategy understood.
- API documented in OpenAPI.
- Operational logs/metrics added for failures and important background jobs.
- Error and empty/loading states implemented in UI.
- No secrets/hard-coded site IDs/SLA values in source.
- Peer code review complete.
- DEV/UAT evidence attached to the story.

25. Fresher Developer Guardrails — Common Mistakes to Avoid

Do Not	Do This Instead
Store incident history only in the current incident row	Keep current state plus append-only incident_events/audit_events.
Let UI change status freely	Use backend transition commands with validation.
Hard-code P1/P2 time values in code	Read versioned SLA policies from database/config.
Create a ticket for every monitoring event	Fingerprint, correlate and deduplicate.
Put all hardware vendor payloads into the main schema	Normalize common fields; retain limited JSONB/reference for vendor extras.

Do Not	Do This Instead
Store millions of metrics/logs in Postgres	Use monitoring/log platforms; store only latest state and incident evidence.
Expose iLO/iDRAC directly	Use a site collector and outbound TLS.
Use local browser clock for SLA	Calculate time server-side with UTC + policy timezone/calendar.
Delete records to “clean data”	Retire/cancel and preserve audit/history.
Build AI first	Build reliable data, workflows and integrations first.
Create many microservices	Use modular monolith until real scaling boundaries appear.
Log request bodies blindly	Mask secrets and sensitive data.

26. Monitoring, Support and Operational Runbooks for the Platform Itself

- Monitor API availability, error rate, latency and CPU/memory.
- Monitor Postgres connections, disk, replication/backup status and slow queries.
- Monitor Redis availability, queue depth, stuck/failed jobs.
- Monitor collector heartbeat per site and last successful poll.
- Monitor each external integration separately; one vendor outage must not mark the entire platform down.
- Alert on SLA job failure, notification backlog and event-ingestion rejection spikes.
- Maintain runbooks for database restore, Redis failure, collector reinstall, certificate rotation and vendor API credential rotation.

27. Future Roadmap — Only After Core Stability

- AI-assisted incident summarization and shift handover.
- Similar incident / known-error recommendation.
- Log/alert correlation with probable root-cause suggestions.
- Predictive hardware risk based on historical health and failure data.
- Automated vendor entitlement/warranty synchronization where APIs and contracts permit.
- Mobile/PWA on-call workflow and push notifications.
- Advanced service dependency maps and blast-radius analysis.
- Capacity and trend analytics for storage, power and network utilization.

AI rule — AI may assist diagnosis and summarization but must not silently change incident priority, execute infrastructure commands, close major incidents, or create/approve changes without explicit policy and human control.

28. MOM-to-Product Traceability

MOM / Operating Need	Product Capability	Build Phase
24x7 support model	Support calendars, on-call roster, ownership and escalation	1-2
Accurate availability / no unrealistic “0 downtime” promise	Health, risk, BCP, availability reporting	2-5
Separate JSAN and CTS RACI	managed_by, support groups, site-scoped roles/views	1

MOM / Operating Need	Product Capability	Build Phase
Incident ownership by type	Category + CI type routing rules	1-2
Escalation matrix	SLA threshold jobs and escalation policies	2
Support group DL / reporting point	Email/API intake + support groups	1-2
Business-hours vs emergency SLA	Versioned calendars and priority policies	2
Site-specific SOPs	Knowledge/runbook module	5
Ticketing tool	Incident/request modules	1
Monitoring tool + ticket integration	Alert pipeline + monitoring adapters	3
Site contacts	Site contact registry / on-call	1
Local infrastructure support	Vendor/local provider records and worklogs	5
Vendor management	Vendor cases, contracts, warranty, RMA	4-5
Infrastructure risk action plan	Risk register linked to sites/CIs	5
BCP confirmation	BCP plans, RTO/RPO, test evidence	5

29. Sample Incident Page — Required Information Order

1. Header: incident number, status, priority, site, affected CI, owner, SLA countdown/risk.
2. Current impact: concise technical/business effect and latest health.
3. Actions: acknowledge, assign, transition, add worklog, add vendor case, resolve.
4. Timeline: monitoring events, user updates, owner changes, comments, worklogs, vendor/RMA updates and SLA events in one chronological stream.
5. Technical context: CI identity, manufacturer/model, service tag/serial, health components, last heartbeat, linked logs/monitoring.
6. Relationships: related alerts/incidents/problem/change/maintenance/vendor case.
7. Resolution/RCA: resolution category, restored_at, root-cause summary and follow-up actions.
8. Audit: protected view for authorized roles.

30. Initial Product Owner Decisions Needed Before Production

Decision	Why It Matters
Confirmed site list and ownership	Controls RBAC, routing and RACI views
P1-P4 impact/urgency matrix	Determines priority consistently
Support hours per site/service	Required for SLA clock
Acknowledgement and resolution targets	Required for SLA policy
Which pending statuses pause which SLA	Prevents disputes and incorrect reporting
Escalation contacts and levels	Required for automated escalation
Dell/HPE tooling/credentials available at each site	Determines integration method
Monitoring platform to standardize first	Avoids parallel adapter development
Log platform and retention policy	Controls storage, security and cost
CTS visibility level and internal-note rules	Required for data separation

Decision	Why It Matters
Data retention and evidence policy	Affects database/object/log architecture
Production hosting/security standard	Required for final deployment architecture

31. Recommended First Development Demo

The first meaningful demo should be a vertical slice, not a collection of disconnected screens.

1. Admin creates one site, rack and two server CIs.
2. A service desk user manually creates an incident linked to Server 1.
3. The backend calculates priority and starts a configured SLA.
4. Engineer acknowledges, clocks 20 minutes of remote work, adds an internal note and changes status to In Progress.
5. A simulated Redfish “disk predictive failure” event is ingested and appears on the same incident/CI timeline.
6. Engineer creates a simulated HPE/Dell vendor case and records an RMA replacement.
7. A simulated recovery event changes CI health back to Healthy.
8. Engineer resolves the incident with restoration evidence.
9. Manager opens the dashboard and sees updated site health, SLA result, vendor timeline and engineer effort.
10. Auditor opens the audit view and can reconstruct every action.

If this vertical slice works correctly — the team has proven the essential architecture: identity → CMDB → ticket → SLA → worklog → telemetry → vendor → audit → reporting.

Appendix A — Glossary

Term	Meaning
ITSM	IT Service Management — structured management of incidents, requests, changes, problems and service quality.
CMDB	Configuration Management Database — authoritative model of managed infrastructure and relationships.
CI	Configuration Item — a managed asset/service/component represented in the CMDB.
NOC	Network/Operations Center — team monitoring and responding to operational issues.
SLA	Service Level Agreement / target — response or resolution timer governed by policy/calendar.
MTTA	Mean Time to Acknowledge.
MTTR	Mean Time to Restore/Resolve, depending on defined metric.
RCA	Root Cause Analysis.
RMA	Return Merchandise Authorization / hardware replacement lifecycle.
Redfish	Standard RESTful interface widely used for server hardware management.
iDRAC	Dell server remote management controller.
iLO	HPE server remote management controller.
OME	Dell OpenManage Enterprise.

Term	Meaning
SNMP	Protocol commonly used to monitor network/power devices.
BCP	Business Continuity Plan.
RTO	Recovery Time Objective.
RPO	Recovery Point Objective.
Idempotency	Processing the same external request/event more than once produces one logical outcome rather than duplicates.

Appendix B — Architecture Decision Records to Create

ADR	Decision
ADR-001	Use a modular monolith for core OpsDesk v1.
ADR-002	Use PostgreSQL as the operational system of record.
ADR-003	Keep raw time-series metrics and high-volume logs outside the OpsDesk database.
ADR-004	Use a secure outbound site collector for infrastructure management integration.
ADR-005	Use Redfish as the normalized server hardware integration baseline.
ADR-006	Use versioned, configurable SLA policies and support calendars.
ADR-007	Use append-only incident/audit events for operational traceability.
ADR-008	Do not permit remote destructive infrastructure actions in v1.

END OF BUILD SPECIFICATION